

浙江大学实验报告

专业:

姓名:

学号:

日期:

地点:

课程名称: 数字信号处理 指导老师: 齐小康 实验名称:
实验类型: 验证型 成绩: 签名:

1 实验目的

本实验结合理论教材中有关快速傅里叶变换(FFT)的教学内容,学习和掌握按时间抽取的基-2FFT 算法原理和实现方法。

2 实验设备/仪器

1) 可运行 MATLAB R2022b 版本的笔记本一台

3 实验原理

长度为 N 的有限长序列 $x(n)$ 的 DFT 为

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}, k = 0, 1, 2, \dots, N-1 \quad (8-1)$$

考虑到 $x(n)$ 一般为复数序列的情况,按照式(8-1)计算每一个 $X(k)$ 值需要 N 次复数乘法, $N-1$ 次复数加法。为了减少计算量,考虑把 N 点 DFT 分解成几个较短的 DFT,并利用因子 W_N^k 的周期性、对称性进行合并归类处理,减少 DFT 运算次数。基于这一思想提出的快速算法称为快速傅里叶变换(Fast Fourier Transform)算法,又称为 FFT 算法。

基 2-FFT 算法基本上分为两大类:时域抽取法 FFT(简称 DIT-FFT)和频域抽取法 FFT(简称 DIF-FFT),下面简要介绍其原理。

8.2.1 时域抽取法 FFT(DIT-FFT)算法原理

设序列 $x(n)$ 的长度为 N ,且满足 $N=2^M$, M 是自然数。如果序列的长度不满足这个条件,可以在序列末尾补零使之满足。将序列 $x(n)$ 按 n 为奇、偶数分为 $x_1(n)$ 、 $x_2(n)$ 两组长度为 $\frac{N}{2}$ 的子序列。

$$\begin{cases} x_1(r) = x(2r) \\ x_2(r) = x(2r+1) \end{cases} \quad r = 0, 1, \dots, \frac{N}{2} - 1 \quad (8-2)$$

用 $\frac{N}{2}$ 点 $X_1(k)$ 和 $X_2(k)$ 表示序列 $x(n)$ 的 N 点 DFT $X(k)$, 其中 $X_1(k)$ 和 $X_2(k)$ 分别表示序列 $x_1(n)$ 、 $x_2(n)$ 的 $\frac{N}{2}$ 点 DFT。

$$\begin{aligned} X(k) &= \sum_{n=\text{偶数}} x(n) W_N^{nk} + \sum_{n=\text{奇数}} x(n) W_N^{nk} = \sum_{r=0}^{\frac{N}{2}-1} x(2r) W_N^{2rk} + \sum_{r=0}^{\frac{N}{2}-1} x(2r+1) W_N^{(2r+1)k} \\ &= \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_N^{2rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_N^{2rk} = \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_{\frac{N}{2}}^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_{\frac{N}{2}}^{rk} \\ &= X_1(k) + W_N^k X_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \end{aligned} \quad (8-3)$$

由于 $X_1(k)$ 和 $X_2(k)$ 均以 $\frac{N}{2}$ 为周期, 且 $W_N^{k+\frac{N}{2}} = -W_N^k$, 所以 $X(k)$ 又可表示为

$$\begin{cases} X(k) = X_1(k) + W_N^k X_2(k) \\ X\left(k + \frac{N}{2}\right) = X_1(k) - W_N^k X_2(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (8-4)$$

这样就将一个 N 点 DFT 计算分解成了两个 $\frac{N}{2}$ 点 DFT 计算。式(8-4)可以用一个蝶形运算流图符号来表示, 如图 8.1 所示。

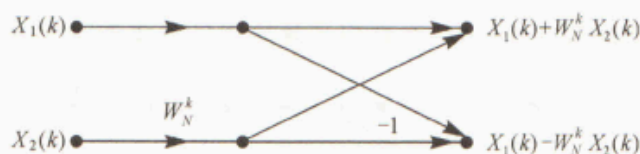


图 8.1 按时间抽取的蝶形运算流图符号

可以看出, 每个蝶形运算需要一次复数乘法和两次复数加法。对比直接按照 DFT 定义式计算全部的 $X(k)$ 需要的 N^2 次复数乘法, $N(N-1)$ 次复数加法, 采用 FFT 算法计算全部的 $X(k)$ 的运算量是 $\frac{MN}{2}$ 次复数乘法和 MN 次复数加法, 当 N 的数值较大时, 可以大大降低计算的复杂度。为了进一步降低运算量, 可以继续对两个子序列 $x_1(n)$ 和 $x_2(n)$ 按照 n 为奇数、偶数继续分解, 直至最后分解的子序列只包含两个元素为止。

8.2.2 频率抽取法 FFT(DIF-FFT)算法原理

设序列 $x(n)$ 的长度为 N , 且满足 $N=2^M$, M 是自然数。如果序列的长度不满足这个条件, 可以在序列末尾补零使之满足。将序列 $x(n)$ 按前后对半分为 $x_1(n)$ 、 $x_2(n)$ 两组长度为 $\frac{N}{2}$ 的子序列。

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=\frac{N}{2}}^{N-1} x(n) W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=0}^{\frac{N}{2}-1} x\left(n + \frac{N}{2}\right) W_N^{\left(n+\frac{N}{2}\right)k} = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + W_N^{\frac{N}{2}k} x\left(n + \frac{N}{2}\right) \right] W_N^{nk} \end{aligned}$$

因为 $W_N^k = (-1)^k = \begin{cases} 1, & k \text{ 是偶数} \\ -1, & k \text{ 是奇数} \end{cases}$, 将 $X(k)$ 分成偶数组和奇数组。当 k 取偶数 $k=2r$ ($r=0, 1, \dots, \frac{N}{2}$) 时, 有

$$X(2r) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_N^{2rm} = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_N^m \quad (8-5)$$

记 $x_1(n) = x(n) + x\left(n + \frac{N}{2}\right)$; 当 k 取奇数 $k=2r+1$ ($r=0, 1, \dots, \frac{N}{2}$) 时, 有

$$X(2r+1) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^{(2r+1)n} = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n W_N^m \quad (8-6)$$

记 $x_2(n) = \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n$ 。将 $x_1(n)$ 、 $x_2(n)$ 分别代入式(8-5)和式(8-6)可得

$$\begin{cases} X(2r) = \sum_{n=0}^{\frac{N}{2}-1} x_1(n) W_N^m \\ X(2r+1) = \sum_{n=0}^{\frac{N}{2}-1} x_2(n) W_N^m \end{cases} \quad r = 0, 1, \dots, \frac{N}{2} - 1 \quad (8-7)$$

将序列 $x_1(n)$ 、 $x_2(n)$ 和 $x(n)$ 用一个蝶形运算流程图符号来表示, 如图 8.2 所示。

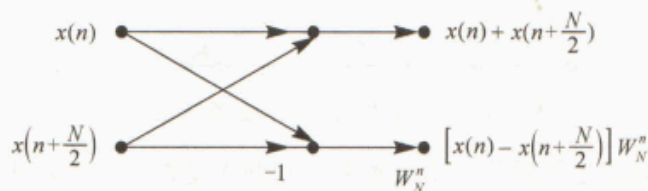


图 8.2 按频率抽取的蝶形运算流程图符号

8.2.3 FFT 算法的特点

无论是按时间抽选的快速算法(DIT-FFT), 还是按频率抽选的快速算法(DIF-FFT), 都具有以下特点。

(1) 蝶形结构、运算量小

DIT-FFT 的运算以如图 8.1 所示的蝶形结构为基本运算单元, DIF-FFT 的运算以如图 8.2 所示的蝶形结构为基本运算单元, 运算结构则由这些蝶形运算结构组合而成。若序列 $x(n)$ 的长度满足 $N=2^M$, 则一共需分解成 M 级运算, 每级包含 $\frac{N}{2}$ 个蝶形运算, 总的蝶形运算个数为 $\frac{MN}{2}$ 个。

复乘次数:

$$m_F = 1 \times \frac{N}{2} M = \frac{N}{2} \log_2 N \quad (8-8)$$

$$\text{复加次数:} \quad a_F = 2 \times \frac{N}{2} M = NM = N \log_2 N \quad (8-9)$$

(2) 采用原位运算

蝶形计算具有进行原位(或同址)计算的优点。每一级的蝶形输入和输出在运算前后可以存储在同一地址的存储单元中,所以只需要 N 个存储单元。

(3) 输入或输出序列的倒位序

DIT-FFT 算法的输入序列为倒位序排列,输出为自然顺序排列。DIF-FFT 算法的输入序列为自然顺序排列,输出为倒位序排列。

8.2.4 离散傅里叶逆变换(IDFT)的快速算法

FFT 算法同样可以用于离散傅里叶逆变换(IDFT)的快速计算,即快速傅里叶逆变换,简称为 IFFT。

由于 IDFT 公式为

$$x(n) = \text{IDFT}[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad (8-10)$$

比较式(8-1)和式(8-10),可以看出,只要把 DFT 公式中的系数 W_N^{nk} 改为 W_N^{-nk} ,并乘以系数 $\frac{1}{N}$,则按时间抽选或按频率抽选的 FFT 都可以用来运算 IDFT。

在 IFFT 计算中,通常把常量 $\frac{1}{N}$ 分解成 M 个 $\frac{1}{2}$ 连乘,即 $\frac{1}{N} = \left(\frac{1}{2}\right)^M$,并且在 M 级的迭代运算中,每级的运算都分别乘上一个 $\frac{1}{2}$ 因子。因此,利用式(8-1)和式(8-10)之间的关系,只需要稍稍改动 FFT 的程序和参数就能实现 FFT 算法。为了完全不用改变 FFT 的程序就可以计算 IFFT,可采用如下方法来实现。首先对 IDFT 式(8-10)取共轭得

$$x^*(n) = \frac{1}{N} \sum_{k=0}^{N-1} X^*(k) W_N^{nk} \quad (8-11)$$

因而有

$$x(n) = \frac{1}{N} \left[\sum_{k=0}^{N-1} X^*(k) W_N^{nk} \right]^* = \frac{1}{N} \{ \text{DFT}[X^*(k)] \}^* \quad (8-12)$$

这说明,只要先将 $X(k)$ 取共轭,就可以直接利用 FFT 子程序,最后再将运算结果取一次共轭,并乘以 $\frac{1}{N}$,即得到 $x(n)$ 值。因此,FFT 运算和 IFFT 运算就可以共用一个子程序。

4 实验内容和结果

4.1. 课本示例程序

4.1.1. 例一

已知序列 $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8\}$, 编程实现 DIT-FFT, 计算 $X(k)$

```

1 %% Experiment 8 -1, ditfft
2 xn=[1 2 3 4 5 6 7 8];
3 Xk=ditfft(xn);
4 disp(Xk);

```

图 5: 例 1 程序示例代码

代码运行结果如下:

```

>> test
列 1 至 7
36.0000 + 0.0000i -4.0000 + 9.6569i -4.0000 + 4.0000i -4.0000 + 1.6569i -4.0000 + 0.0000i -4.0000 - 1.6569i -4.0000 - 4.0000i
列 8
-4.0000 - 9.6569i

```

图 6: 例 1 程序示例代码运行结果

4.1.2. 例二

已知序列的频谱 $X(k)$ 为 $\{36.0000, -4.0000 + 9.6569i, -4.0000 + 4.0000i, -4.0000 + 1.6569i, -4.0000, -4.0000 - 1.6569i, -4.0000 - 4.0000i, -4.0000 - 9.6569i\}$, 试求序列 $x(n)=\text{IDFT}[X(k)]$ 。

```

1 %% Experiment 8-2
2 clc;clear;close all;
3 Xk=[36,-4+9.6569i,-4+4i,-4+1.6569i,-4,-4-1.6569i,-4-4i,-4-9.6569i];
4 N=length(Xk);
5 Xk1=conj(Xk);
6 xn1=ditfft(Xk1);
7 xn1=conj(xn1)/N;
8 xn1=real(xn1);
9
10 xn2=fft(Xk1);
11 xn2=conj(xn2)/N;
12 xn2=abs(xn2);
13 disp(xn1);
14 disp(xn2);

```

图 7: 例 2 程序示例代码

代码运行结果如下:

```

1.0000    2.0000    3.0000    4.0000    5.0000    6.0000    7.0000    8.0000

1.0000    2.0000    3.0000    4.0000    5.0000    6.0000    7.0000    8.0000

```

图 8: 例 2 程序示例代码运行结果

4.1.3. 例三

对序列进行离散傅里叶变换 (DFT) 时, 试比较采用直接计算 DFT 与采用快速算法 FFT 计算的时间差异。

```

1  %% Experiment 8-3
2  clc;clear;close all;
3  Nmax = 8001;          % 增加N的长度，方便评估计算时间；
4  %-----DiT-FFT
5  ditfft_time = zeros(1,Nmax);
6  for n = 1:50:Nmax
7      x = rand(1,n);
8      t = clock;
9      ditfft(x);
10     ditfft_time(n) = etime(clock,t);
11 end
12 k = 1:50:Nmax;
13 subplot(3,1,1);plot(k,ditfft_time(k),'r-o');hold on;
14 axis([0 Nmax 0 max(ditfft_time)]);
15 ylabel('t/s');title('Dit-FFT执行时间');
16 %----DFTfor
17 DFTfor_time = zeros(1,Nmax);
18 for n =1:800:Nmax
19     x = rand(1,n);
20     t = clock;
21     DFTfor(x);
22     DFTfor_time(n)=etime(clock,t);
23 end
24 k=1:800:Nmax;
25 subplot(3,1,2);plot(k,DFTfor_time(k),'r-o'); hold on;
26 axis([0 Nmax 0 max(DFTfor_time)]);
27 ylabel('t/s');title('DFTfor执行时间');
28 %---DFTmat
29 DFTmat_time = zeros(1,Nmax);
30 for n =1:800:Nmax
31     x = rand(1,n);
32     t = clock;
33     DFTfor(x);
34     DFTmat_time(n)=etime(clock,t);
35 end
36 k=1:800:Nmax;
37 subplot(3,1,3);plot(k,DFTmat_time(k),'r-o');hold on;
38 axis([0 Nmax 0 max(DFTmat_time)]);
39 ylabel('t/s');title('DFTmat执行时间');

```

图 9: 例 2 程序示例代码

代码运行结果如下：

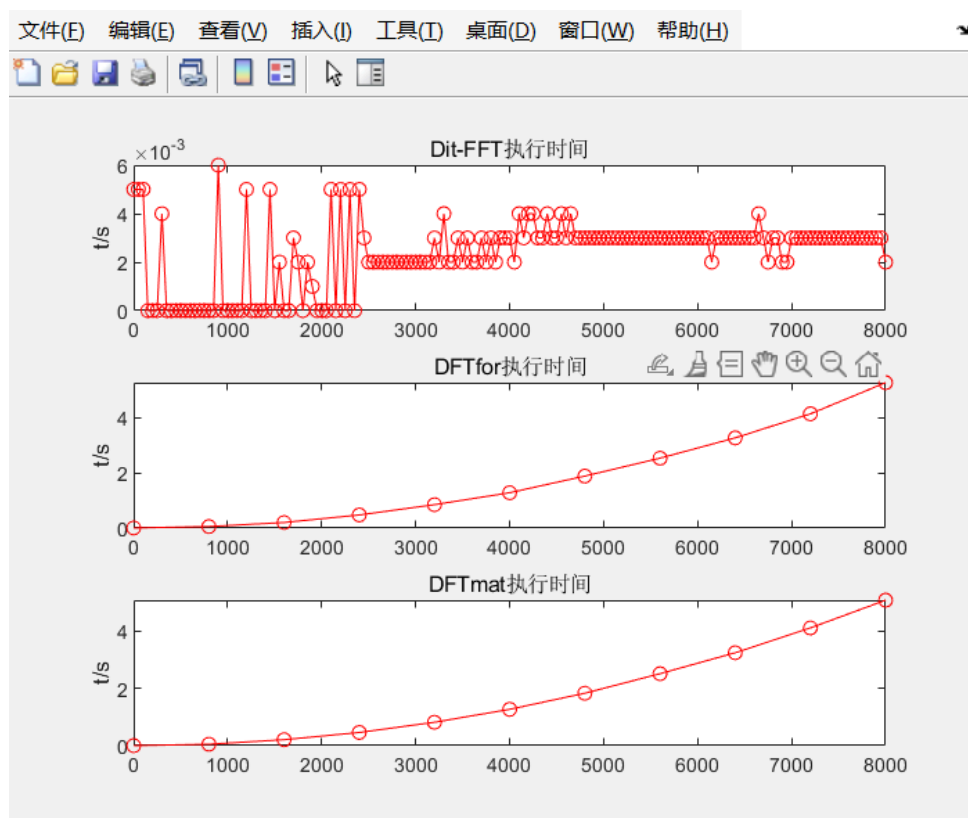


图 10: 例 2 程序示例代码运行结果

4.2. 课后实验

已知序列 $x(n)$ 的 DFT 为 $\{36.0000, -4.0000 + 9.6569i, -4.0000 + 4.0000i, -4.0000 + 1.6569i, -4.0000, -4.0000 - 1.6569i, -4.0000 - 4.0000i, -4.0000 - 9.6569i\}$, 模仿例题按图 8.7 所示流程图编程实现 IFFT 功能计算 $x(n)$, 并与直接调用函数 `ifft()` 命令对比计算结果是否正确。

```

1  %% Experiment 8-3
2  clc;clear;close all;
3  Nmax = 8001;          % 增加N的长度，方便评估计算时间；
4  %-----DiT-FFT
5  ditfft_time = zeros(1,Nmax);
6  for n = 1:50:Nmax
7      x = rand(1,n);
8      t = clock;
9      ditfft(x);
10     ditfft_time(n) = etime(clock,t);
11 end
12 k = 1:50:Nmax;
13 subplot(3,1,1);plot(k,ditfft_time(k),'r-o');hold on;
14 axis([0 Nmax 0 max(ditfft_time)]);
15 ylabel('t/s');title('Dit-FFT执行时间');
16 %----DFTfor
17 DFTfor_time = zeros(1,Nmax);
18 for n =1:800:Nmax
19     x = rand(1,n);
20     t = clock;
21     DFTfor(x);
22     DFTfor_time(n)=etime(clock,t);
23 end
24 k=1:800:Nmax;
25 subplot(3,1,2);plot(k,DFTfor_time(k),'r-o'); hold on;
26 axis([0 Nmax 0 max(DFTfor_time)]);
27 ylabel('t/s');title('DFTfor执行时间');
28 %---DFTmat
29 DFTmat_time = zeros(1,Nmax);
30 for n =1:800:Nmax
31     x = rand(1,n);
32     t = clock;
33     DFTfor(x);
34     DFTmat_time(n)=etime(clock,t);
35 end
36 k=1:800:Nmax;
37 subplot(3,1,3);plot(k,DFTmat_time(k),'r-o');hold on;
38 axis([0 Nmax 0 max(DFTmat_time)]);
39 ylabel('t/s');title('DFTmat执行时间');

```

图 11: 课后实验程序代码

代码运行结果如下：

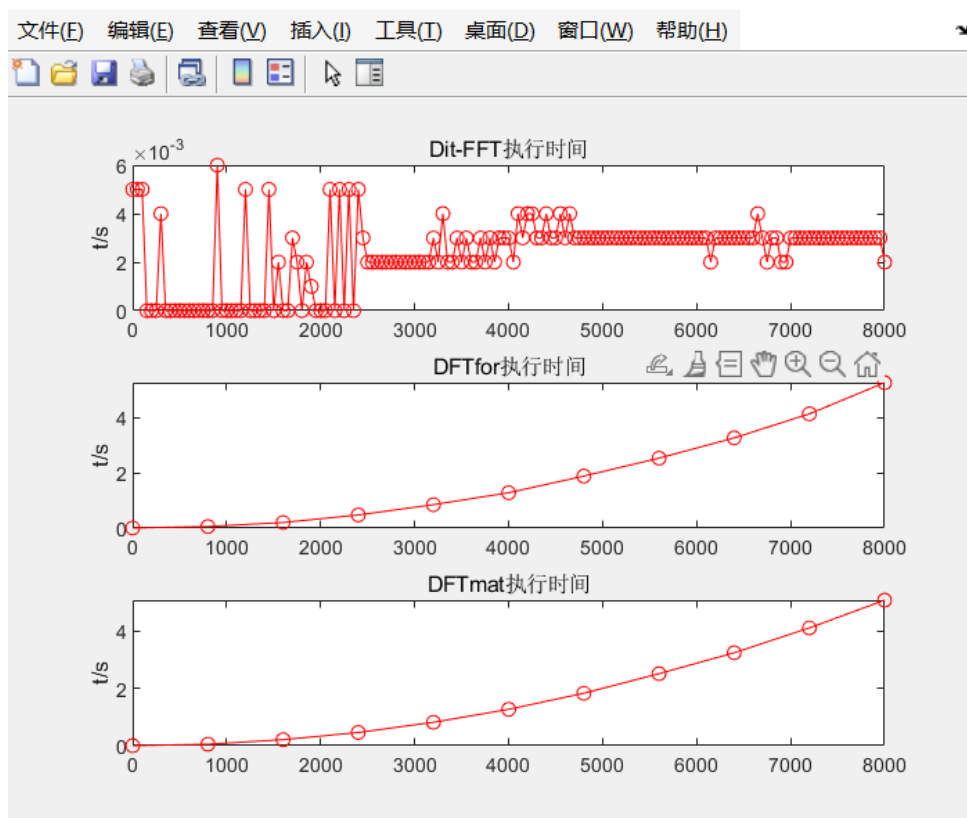


图 12: 课后实验程序代码运行结果

5 实验收获

通过本次实验，我从理论走向实践，对快速傅里叶变换（FFT）有了更扎实的理解。动手编程实现基-2 按时间抽取的 FFT 算法，使我真正弄懂了蝶形运算的每一步流程，将书本上抽象的流程图变成了清晰的代码逻辑。在成功实现了 FFT 之后 T 之后，我又通过修改旋转因子符号并添加归一化步骤，进一步完成了 IFFT 的实现，并将其结果与 MATLAB 内置函数进行比对验证。这一过程让我深刻体会到 FFT 与 IFFT 之间的内在对称性与互逆关系，而不仅仅是记住两个对立的公式。实验中对直接计算 DFT 与 FFT 的运行时间，让我直观感受到后者在大数据量下的巨大效能优势，理解了其作为现代数字信号处理基石的意义。整个过程中遇到的诸如旋转因子符号错误、忘记归一化等问题，也锻炼了我调试程序和严谨分析的能力。总的来说，这次实验让我将对 FFT 的认识从一个高效的数学概念，落实为一个具体、可控并能亲自复现的强大工具。